

Project 06 — Drone Infrastructure Inspection on LiteRT

IoT / edge · onboard aerial analysis · compute measured in flight minutes

Contents

Project 06 — Drone Infrastructure Inspection (LiteRT)	1
0. Numbered build plan	1
1. Brief	2
2. Why LiteRT for this workload	2
3. No KV cache here — the substitution	3
4. Models and compression	3
5. Architecture	4
6. Frontend	4
7. Architecture graph: node inventory	5
8. The improvement loop for this project	6
8a. LoRA adapters: where they earn their place here	6
9. The release gate for this project	7
10. Monitoring and KPIs	8
11. Security, audit and compliance	8
12. Deployment	9
13. Deliverables	9
14. Out of scope	9

Project 06 — Drone Infrastructure Inspection (LiteRT)

Read `00-shared-requirements.md` first. Everything in it applies here, with the device-class substitutions in §4.

0. Numbered build plan

Paste this section into Claude Code’s plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **LiteRT serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §7 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a “Say this” line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.

7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest source badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §8. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §8a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §9 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging → production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §13 of this document plus the shared deliverables checklist.

1. Brief

Build an **onboard aerial inspection system** for infrastructure: solar farms, transmission lines, roofs, wind turbine blades. A drone flies a survey pattern; a companion computer analyses each frame in flight, flags defects — cracked or soiled panels, thermal hotspots, corrosion, vegetation encroaching on a line — geotags them, and produces a prioritised report before the aircraft has landed.

The constraint that makes this project unlike the others in the pack:

Compute costs flight time. Every watt the inference board draws is a watt not driving the rotors. On a 25-minute airframe, a 10 W processing load can cost two to three minutes of endurance — which is several hundred panels not surveyed per flight.

Nowhere else in this pack does model efficiency convert so directly into a business number. Optimisation here is not about cloud spend; it is about how much asset gets inspected per battery. That framing should run through the entire client-facing narrative.

Target user: utility-scale solar operators, transmission network owners, industrial roof surveyors, wind operators — anyone currently paying for manual inspection or reviewing thousands of aerial photos by hand.

2. Why LiteRT for this workload

LiteRT (formerly TensorFlow Lite) is built for exactly this envelope:

- **Full-integer INT8 quantization** producing models that run on EdgeTPU-class accelerators. A Coral EdgeTPU draws about 2 W and delivers several TOPS — the best inference-per-watt available in this form factor, and on a drone that ratio is the whole engineering problem.
- **Delegate ecosystem** — EdgeTPU, GPU, NNAPI, XNNPACK — so the same `.tflite` runs on a Coral, a Pi GPU, or bare ARM CPU with a graceful fallback. Useful because airframe payload options change and the software should not.
- **Small runtime.** The interpreter is a few hundred kilobytes, which matters on a constrained companion computer.
- **Model Optimization Toolkit** gives quantization-aware training, **pruning** and weight clustering in one pipeline. This is the one project in the pack where pruning genuinely earns its place, because the compute budget is hard-capped by physics rather than by cost.

The honest constraint: full-integer quantization for EdgeTPU is strict. Unsupported operators fall back to CPU and destroy the latency budget silently. Verify the compiled model has no CPU fallback segments, and make that a build-time check rather than something discovered in flight.

3. No KV cache here — the substitution

Per shared requirements §4:

A defect detector runs **one forward pass per captured frame**. There is no token generation, no KV state, no KV cache, and no continuous batching in the LLM sense.

What is demonstrated instead — and this project has the richest substitution story in the pack because the power budget forces every lever to be used:

Technique	What it buys
Full-integer INT8 quantization (QAT)	Required for EdgeTPU; ~4× smaller, order-of-magnitude better inference/watt
Structured pruning	Fewer FLOPs and less memory traffic — directly fewer joules per frame
Weight clustering	Smaller artifact, faster OTA updates over a field link
Delegate selection	EdgeTPU where fitted, GPU or XNNPACK fallback
Tiled inference at native resolution	Aerial defects are small; downscaling a 20 MP frame destroys them. Tile instead, and only tile where a coarse pass found something
Two-tier triage	Cheap onboard screening, expensive analysis on the ground
Adaptive capture rate	Slow the frame rate over uniform terrain, raise it over structures

The two-tier design is the key architectural idea. Onboard, run a small, fast model that answers “is there anything here worth keeping?” — discarding the 90%+ of frames showing intact panels. Store and later process only the candidates on a ground station with a larger model. This cuts onboard compute, onboard storage and post-flight review time simultaneously, and it is the decision a client should understand from the architecture graph.

4. Models and compression

Stage	Model	Where
Onboard triage	MobileNetV3-Small or EfficientDet-Lite0, INT8	Companion computer
Onboard detection	EfficientDet-Lite1 / YOLO-nano, INT8	Companion computer
Ground analysis	Larger segmentation model for defect extent	Ground station / laptop
Thermal (if fitted)	Small CNN on radiometric data	Onboard

Compression spec - Quantization-aware training, not post-training quantization, for the onboard models. Aerial defects are small, low-contrast features; PTQ tends to lose exactly those while leaving aggregate metrics looking acceptable. - Apply structured pruning, then QAT, then compile for EdgeTPU. Record at each step: model size, FLOPs, inference latency, **watts**, and accuracy. - **Verify zero CPU fallback** in the compiled EdgeTPU model, as a build gate.

Quality gate - Defect detection recall at a fixed precision, gated against the FP32 baseline. - **Recall on small defects specifically** — a hairline crack at 40 m altitude is a handful of pixels, and it is the whole point of the product. Aggregate mAP will not show its loss. - **Inference energy per frame** as a gated metric. If a model change costs flight time it must fail the build, exactly as an accuracy regression would. This is the gate that makes the project’s core constraint enforceable rather than aspirational. - False positive rate: an inspection that flags every panel is worthless, because a human still reviews everything.

5. Architecture

Camera / thermal sensor on airframe

- Companion computer (Raspberry Pi / Jetson Orin Nano + Coral)
- Device authentication (signed boot, mTLS to ground station)
- Authorization (which survey, which asset)
- Frame capture + adaptive rate control
- LiteRT onboard triage (INT8, EdgeTPU) ← discard uninteresting frames
- LiteRT onboard detection on candidates
- Geotag from flight controller (MAVLink): GPS, altitude, gimbal attitude
- Confidence gate + defect classification
- Local storage: candidates only, with metadata
- Local append-only audit log, hash-chained
 - ... landing / link restored ...
- Ground station: larger model, orthomosaic, defect extent, report
- Prometheus → Grafana (fleet health, model performance)
- Model registry → canary aircraft → fleet rollout ← replaces Kubernetes
 - Fleet OTA (canary aircraft, signed artifacts)

== IMPROVEMENT LOOP (the RLHF analogue) =====

Confirm / reject a finding – the inspector does this anyway

Severity re-grading – model said minor, inspector said urgent

Missed defect – added on review, rare and expensive

- store the TILE the model saw, with its geo-reference
- record WHICH inspector: severity is expert judgement
- Retrain (operator-run, never automatic)
- Promotion gate: per-class IoU + recall-on-severe (absolute floor) + energy-per-frame harness

==> back to the LiteRT model

Build deliberately:

- **Geotagging accuracy is the deliverable.** “There is a defect” is not actionable; “there is a defect on panel row 14, string 7, at these coordinates” sends a technician to the right place. Fuse detection with GPS, altitude and gimbal attitude, and validate the projection against known ground points. A detection with a bad position is worse than no detection because it wastes a site visit.
- **The aircraft flies without a link.** All onboard decisions are local. Telemetry and results sync opportunistically; nothing waits on connectivity.
- **Storage is bounded and must degrade gracefully.** When the card fills mid-survey, drop lowest-confidence candidates first rather than stopping capture.
- **Never take flight-critical action.** This is an inspection payload, strictly advisory. It does not steer, does not trigger avoidance, and has no authority over the flight controller. Say this explicitly in the architecture graph — it is the first question any aviation-literate client asks, and the boundary must be structural.

6. Frontend

Product tab — a survey view: map with the flight path, defect pins coloured by severity, and a click-through to the source frame with the detection overlay. A prioritised defect list exportable as an inspection report. **A live in-flight view** showing frames analysed, candidates kept, and — prominently — **watts drawn and estimated flight time remaining**.

That power readout is this project’s equivalent of the KV-cache gauge in the vLLM project: the live, visible manifestation of the core engineering constraint, and the thing to point at on camera.

Configuration: asset type and defect classes, confidence thresholds, capture rate policy, triage aggressiveness with the compute/recall trade shown as a curve.

Architecture tab — 3D graph. Logos: LiteRT/TensorFlow, Coral EdgeTPU, Raspberry Pi, MAVLink/PX4, Prometheus, Grafana. Strongest copy on **INT8 quantization, pruning, two-tier triage, EdgeTPU delegation, adaptive capture**, and the **no-KV-cache** node explaining the substitution.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §7 for the node inventory and `00-shared-requirements.md` §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.



7. Architecture graph: node inventory

These are the stage cards for this project’s Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent’s card. Build them against the `ArchNode` shape in `00-shared-requirements.md` §8a, with all four rationale fields and a “Say this” line on each.

#	Stage card	Colour role	Expands into
0	Flight	neutral	— (the mission, and the asset being inspected) Mission planner, imagery review, finding triage, export
1	Ground Console	teal	
2	Ingest & Security	amber	
3	Capture Pipeline	violet	Link encryption, device identity, authentication, authorization, artifact signature verification Frame capture, exposure and blur rejection, tiling for high-resolution imagery, normalisation
4	LiteRT Runtime	orange	Segmentation / defect model (INT8 full-integer) , NPU / EdgeTPU delegation, structured pruning, delegate fallback path, “no KV cache — and why”
5	Post-processing	violet	NMS, mask assembly, tile stitching, confidence gating, severity classification
6	Geo-referencing	violet	GPS/IMU fusion, tile-to-world mapping, finding localisation
7	Store & Sync	blue	On-board store, deferred sync when the link returns, finding registry

#	Stage card	Colour role	Expands into
8	Monitoring & Audit	magenta	Prometheus, Grafana, hash-chained audit log, flight records, retention policy
9	Improvement Loop	green	Inspector confirmation and rejection of findings, corrected tiles, severity re-grading, retraining set, promotion gate □ segmentation suite, on-device latency and energy harness
10	Fleet OTA	grey	Signed model artifacts, canary aircraft, staged rollout rings, rollback

Two cards carry this project. **LiteRT Runtime**, where the delegate fallback path deserves its own strong copy — an NPU that silently falls back to CPU turns a 30 ms inference into 400 ms and drains the battery mid-flight, and a system that does not detect and report that failure will fail quietly in the field. And **Store & Sync**: an inspection drone works where there is no link, so “deferred sync” is not a nicety, it is the operating mode.

Energy is a first-class metric here in a way it is not anywhere else in this pack: inference that costs battery costs flight time, and flight time is the thing the client is buying.

8. The improvement loop for this project

`00-shared-requirements.md` §5a applies in full. What is specific here:

Inspection has a naturally high-quality feedback signal, because **a human already reviews every finding before it becomes a work order**:

- **Confirm / reject a finding** — the inspector is doing this anyway. Capture it. Zero extra effort, and it is a labelled example.
- **Severity re-grading** — the model said minor, the inspector said urgent. The most valuable signal in the project, because severity drives cost.
- **Missed defect** — added by the inspector on review. Rare, expensive, and the one worth paging about.

Rules for this domain:

- **Store the tile, not the whole frame.** High-resolution imagery is tiled for inference; the training example is the tile the model actually saw, at its actual resolution.
- **Keep the geo-reference with the correction.** A defect label without its location cannot be checked against the asset later, and cross-flight consistency (“did we see this crack last quarter?”) is a large part of the product’s value.
- **Weight by inspector, and record who.** Severity grading is expert judgement and experts disagree; a training set that averages a novice and a twenty-year inspector without distinguishing them loses the expertise.
- **Sync the queue opportunistically**, like everything else on the aircraft. The loop must tolerate days offline.

8a. LoRA adapters: where they earn their place here

Test this — the tiered-model story here makes LoRA unusually attractive.

As with the other vision projects, LiteRT runs a frozen, INT8, delegate-mapped graph: LoRA is a training-time technique, merged before conversion, with no adapter swapping in flight. Keep it in the improvement loop in the architecture graph, never in the onboard serving path.

Where it earns its place: per-asset-class adapters. Solar panels, wind turbine blades, transmission towers, roofs and bridges are different visual problems that share most of a backbone. One model that handles all of them is worse at each than five specialised models — but five full models is five training pipelines, five validation sets and five artifacts to keep in sync.

Approach	Training cost	Accuracy per asset class	Artifacts to manage
One general model	1×	worst	1
Full model per asset class	5×	best	5 independent
Shared backbone + LoRA per class, merged	~1.2×	close to best	5, but one lineage

The onboard model is chosen per mission — the flight plan already knows the asset class — so selection costs nothing at inference time.

The second case: seasonal and regional drift. A defect model tuned on summer imagery degrades on winter imagery: different sun angle, snow, different vegetation. LoRA makes a seasonal re-adaptation cheap enough to actually do each year rather than propose and defer.

Measure: per-class IoU and recall-on-severe for the adapted artifact against the general one, **and energy per frame after merging and re-quantizing.** A merged adapter should not change inference cost — verify that rather than assume it, because a change that quietly moves an operator off the NPU fallback path costs flight time.

9. The release gate for this project

00-shared-requirements.md §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	Held-out inspection suite: per-class IoU / Dice, and recall at the operating threshold , broken down by defect severity	Recall on severe defects is an absolute floor, not a delta. A missed critical defect is the failure that ends the contract; a false positive costs an inspector five minutes. Gate them asymmetrically and say why
Performance	Is it still fast and cheap enough to fly?	On-device harness: p95 inference latency, energy per frame , sustained throughput with thermal soak, and delegate-fallback rate	Energy is flight time. A model that is 10% more accurate and 40% more expensive per frame may be a worse product, and the gate should make that visible rather than let accuracy win by default

lm_eval and GuideLLM do not apply. Substitute the harnesses above, measure on the actual airframe compute with the actual thermal envelope, and **fail the gate if the NPU delegate falls back to CPU at all** — a silent fallback is the single most likely way this system degrades in production.

10. Monitoring and KPIs

Section	Metrics
Quality	Defect recall and precision by defect class and size, false positive rate, geotag positional error, ground-truth agreement on audited samples
Performance	Frames/sec onboard, inference latency, watts and joules per frame, flight minutes consumed by compute , triage discard rate, thermal throttling, storage headroom
Security	Signed boot failures, model integrity failures, unauthorised survey access, link authentication failures
Audit	Every survey: aircraft, operator, model version, area covered, defects found, chain of custody for the report
Improvement	Findings confirmed vs rejected, severity re-grades (by inspector), missed defects reported, tiles awaiting retraining

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

The headline business metric: assets inspected per battery, next to the manual inspection baseline. Everything technical should be traceable to it. A 20% reduction in inference power becomes a directly quotable increase in panels surveyed per flight, and that is the number that closes the sale.

Audit chain of custody matters here more than it might appear: inspection reports feed warranty claims and regulatory compliance. Which model version produced a finding, and whether it can be reproduced, is a question that will be asked when a manufacturer disputes a claim.

11. Security, audit and compliance

- **Aviation regulation is the primary constraint.** The payload must be advisory only, with no authority over flight. Document that boundary explicitly; operators will be asked to demonstrate it.
 - **Signed boot and signed model artifacts.** An aircraft is physically accessible and periodically unattended.
 - **Imagery may capture people and property incidentally.** Even inspecting a solar farm, frames will contain vehicles, roads and occasionally people. Minimise: discard non-candidate frames onboard, and blur or discard incidental subjects in retained frames. State the retention policy.
 - **Airspace and privacy law vary sharply by jurisdiction.** The system should record the survey boundary and flag captures outside it rather than assuming compliance.
 - **Chain of custody** for inspection reports: hash-chained audit log linking each finding to aircraft, flight, model version and operator.
-

12. Deployment

- **Canary aircraft** □ **squadron** □ **fleet** rollout rings for model updates. A bad model that under-detects reaching an entire fleet means a season of surveys that must be repeated.
 - Signed, versioned .tflite artifacts; small artifacts matter because updates go over field links.
 - **Rollback must work without a laptop on site.** Field crews are not engineers.
 - Ground station deployable as a container so a site can run it on a laptop disconnected from anything.
-

13. Deliverables

- Onboard pipeline: capture □ triage □ detect □ geotag □ store
- Ground station: larger-model analysis, map view, inspection report export
- LiteRT pipeline with pruning, QAT, EdgeTPU compilation, and a **no-CPU-fallback** build check
- Quality gate including small-defect recall **and energy per frame**
- All three tabs
- Hash-chained audit log with chain of custody
- Fleet rollout with canary and rollback
- Measured watts, flight-time cost, and detection accuracy on real survey footage

Acceptance: 1. Onboard pipeline sustains the target capture rate within the stated power budget, measured with a real power meter, not a datasheet figure 2. Compiled EdgeTPU model contains zero CPU fallback segments 3. Quality gate fails a model that regresses small-defect recall **or** exceeds the energy budget 4. Geotag positional error within the agreed tolerance against surveyed ground points 5. Full survey completes with no network link 6. Architecture tab and guided tour run offline on the ground station

14. Out of scope

Flight control, autonomy, obstacle avoidance — advisory payload only. Person detection or tracking; incidental subjects are minimised, not analysed. Beyond visual line of sight operations, which carry their own certification burden. Photogrammetry and 3D reconstruction beyond a simple orthomosaic. Real-time video downlink.